COMP64301 Coursework

A Comparative Study of Traditional Computer Vision and Deep Learning in Object Recognition

1. Introduction

Computer vision and deep learning methods play an increasingly important role in the field of robotics. This report focusses on object recognition, using deep learning convolutional neural network model and traditional computer vision methods to analyse two datasets, select CiFAR dataset and SVHN dataset, and discuss the performance of the two methods in object and digital image recognition. The second part of the report expresses the deep learning method based on CNN, the third part expresses the traditional computer vision method, the fourth part compares and analyses two different methods, and the fifth part introduces the latest technology and application of computer vision in the field of robotics.

2. Deep Learning

2.1 Methodology

2.1.1Network

This report adopts two different deep learning models for two different datasets. To adapt the characteristics and complexity, I adjust the parameters of the two datasets. The core residual network width of the CiFAR-10 dataset starts from 64 channels, and is expanded to 256 channels through three convolutional functions, gradually extracting the complex textures and different levels of the body; the core residual network width of the SVHN dataset is expanded from 96 channels to 384 channels, using more filters to reduce the noise caused by the surrounding street scene and better extracts the detailed features of the number. Both of them adopt global average pooling. Because the SVHN dataset is relatively fixed, a multi-scale feature pyramid structure is also added to identify numbers of different sizes through the fusion of different depth features. After the pool layer, the regularization constraint adopts incremental Dropout, but because the SVHN dataset is simpler than CiFAR-10, the risk of overfitting is higher. Therefore, double Dropout is adopted in the full connection layer, the parameter is set to 0.5, and the L2 norm square is added to the loss function, so that the model would not to the extreme.

CIFAR-10	SVHN	Reasons
Strong Spatial Augmentation (rotation, flipping, etc.)	Minor color adjustments after standardization	Numbers require the geometric structure to remain unchanged.
Deep progressive residual network (64→128→256)	Progressive residual network with increasing width (96→192→384)	CiFAR-10 dataset needs to extract semantic features, and SVHN dataset needs to extract detailed features.
Global average pooling	Global average pooling Multi-scale feature pyramid	Identify different distance of the number
Incremental Dropout	Strong Dropout (dual 0.5) + L2 weight decay	SVHN digits require stronger regularization constraints to prevent overfitting.
GAP → Dense (10)	Feature fusion → Dense (512) → Dense (10)	SVHN uses a nonlinear projection layer for dimensionality reduction and feature integration.

Table 2.1 Network for the CIFAR-10 and SVHN Datasets

2.1.2 Hyperparameter

After training the basic CNN model, the accuracy of the SVHN dataset reached 85%, but the CiFAR-10 data set showed only 54% accuracy, and the training efficiency was low, and the training cost was high. Therefore, first of all, change the RMSprop optimizer to the Adam optimizer, quickly pass through the gradient gentle area through order moment estimation, and achieve rapid convergence, and replace flatten with global average pooling to reduce parameters; then adjust the fixed learning rate to dynamic learning rate scheduling based on verification loss, so that the model could reduced automatically during the loss platform period, which solves the problem of slow convergence or shock in the later stage, and the accuracy rate is improved by about 8%. After the parameter adjustment, the training time was shortened and the accuracy rate of the CiFAR-10 dataset was increased from 54% to 70%. The accuracy rate of the SVHN dataset decreased from 85% to 84% when only 14% dataset was trained. Therefore, it can be verified that the parameter adjustment at this stage performed better in both data sets. For the SVHN dataset, dropout, which increases the depth of the network, is adopted, and the abandonment rate is set to 0.5 to cope with the risk of overfitting after the deepening of the network.

In the process of trying model integration, I first tried to adopt simple average integration. By training multiple independently initialized models and averaging their predictions, it is found that the accuracy rate does not change much, but the training time is greatly

increased. Therefore, it is improved and tries to weigh different models according to the accuracy of each model on the verification set. The weight is distributed, but the effect is still not obvious enough, and the cost is high. Finally I chose the model stacking method, but the training time has still increased. After the SVHN dataset test, the accuracy rate was improved to 96%. In future research, the CiFAR dataset can be tried, or a better integration method could be found.

2.1.2 Datasets

Changing the perspective through rotation, translation, flipping, etc. has a good effect on natural images of CIFAR-10, and the accuracy rate has increased from 70% to 78%. However, after the SVHN dataset was tried, the accuracy rate plummeted to 19.74%. After analysis, it was found that enhancement would destroy the structure of the number. For example, the "9" after digital rotation may be mistaken for "6". So a wider network channel and integrated learning were built after using TensorFlow Datasets to load the data set.

2.2 Results

Following final parameter tuning, the test set accuracy of CiFAR reached 78.77%. The training process for the training set is as follows:

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss	Learning Rate	Time Per Epoch
1/10	34.54%	1.9044	55.32%	1.2439	0.001	251
2/10	58.91%	1.1462	62.90%	1.025	0.001	260
3/10	66.13%	0.9511	68.57%	0.8729	0.001	251
4/10	70.30%	0.8441	63.85%	1.023	0.001	285
5/10	73.43%	0.7593	74.52%	0.7236	0.001	299
6/10	75.04%	0.7129	77.05%	0.6565	0.001	297
7/10	77.48%	0.6501	77.12%	0.6676	0.001	326
8/10	78.38%	0.6149	79.14%	0.5943	0.001	258
9/10	79.83%	0.5782	77.41%	0.6669	0.001	253
10/10	81.05%	0.5451	76.00%	0.7012	0.001	256

Table 2.2 Results for the CIFAR-10 Datasets

As Table 2.2 shows, after the model began to train, the verification accuracy rate was significantly higher than the training accuracy rate. The accuracy rate was greatly improved in the initial stage. The loss was the lowest in the eighth cycle, and the accuracy rate of the verification set reached 79.14%. At the same time, the accuracy rate of the test set is 78.77%, which is also closest to the accuracy rate of the verification set of the eighth cycle, so the model should stop training in the eighth cycle. After that, the accuracy

of the ninth and tenth cycles began to decline, and there was a overfitting.

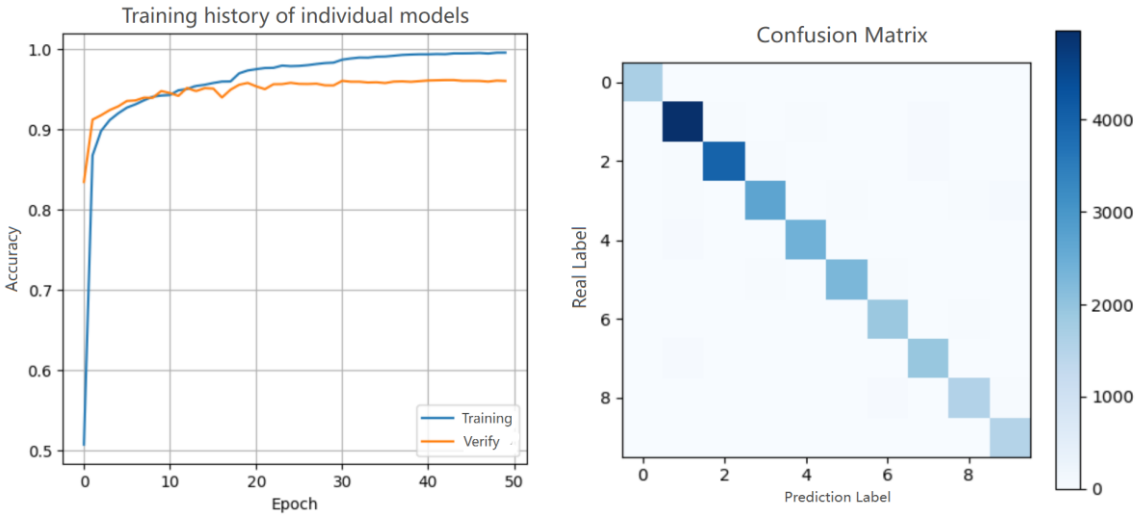

Fig 2.2 Results for the SVHN Datasets

It can also be seen in Fig 2.2 that on the SVHN dataset, this method has obtained good results, and the accuracy rate of the final test set is 96.34%. With the increase in the number of training rounds, the training duration has increased significantly, and the accuracy rate has improved slowly. Therefore, the number of rounds can be reduced to 10 rounds, and the accuracy rate can still reach 94.09%.

3. Traditional Computer Vision

3.1 Methodology

3.1.1Network

The local features of SIFT were extracted from all 26,032 training data sets, and a total of 72,149 words were extracted. Then train the K-Means algorithm and cluster it into a global dictionary, which contains 200 visual words. For each new image, find the nearest visual word through this algorithm. By counting the probability of the appearance of each visual word, a 200-dimensional bag of world model of histograms is obtained. Because the dimension is fixed and the characteristics have formed a numerical word frequency, the SVM is selected as the classifier.

3.1.2 Hyperparameter

In the process of parameter adjustment, I first choose to change the K value to check the change in accuracy rate. Take 50, 100, 120 and 200 respectively to try to evaluate the accuracy rate on the verification set of 5,000 images. The results show that when the K

value increased from 50 to 200, the accuracy rate increased by 13%, but the duration of training increased significantly. If still try to raise the K value again, the cost will be too high, so the K value is set to 200. After determining the K value, choose to adjust the regularization parameters of the SVM, observe the changes in the results, and find that the accuracy rate is the highest when the C value is 1, and the accuracy rate decreases by 12% when the value is 0.01. Therefore, I selected 1 as the regularization parameter. In addition, the SVM of the linear kernel was tried to be replaced with RBF, but it was only 1% higher than the linear kernel on the verification set, and the training time was significantly increased, so the SVM was finally selected as the classifier. For the parameters of SIFT, choose to use the default parameters, set the contrast threshold to 0.04, and set the edge threshold to 10. In additional, as for the SVHN dataset, I tried to add color histograms and HOG features, but the accuracy rate was not significantly improved, and the training time increased significantly, so it was not used.

3.2 Results

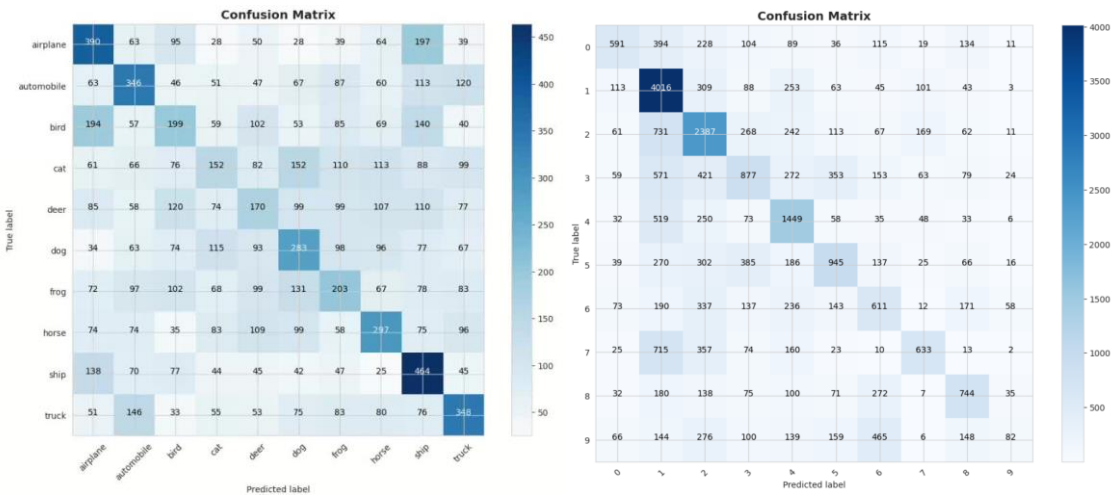

Fig 3.2 Confusion Matrix for the CIFAR-10 and SVHN Datasets

CiFAR-10	Precision	Recall	Support	SVHN	Precision	Recall	Support
airplane	0.34	0.39	993	0	0.54	0.34	1721
automobile	0.33	0.35	1000	1	0.52	0.8	5034
bird	0.23	0.2	998	2	0.48	0.58	4111
cat	0.21	0.15	999	3	0.4	0.31	2872
deer	0.2	0.17	999	4	0.46	0.58	2503
dog	0.28	0.28	1000	5	0.48	0.4	2371
frog	0.22	0.2	1000	6	0.32	0.31	1968
horse	0.3	0.3	1000	7	0.58	0.31	2012
ship	0.33	0.47	997	8	0.5	0.45	1654
truck	0.34	0.35	1000	9	0.33	0.05	1585

Table 3.2 Results for the CIFAR-10 and SVHN Datasets

Traditional methods perform poorly on the CiFAR-10 dataset, achieving only 28.56% accuracy on the test set. Table 3.2 shows higher accuracy for “airplane”, “car”, and “ship”, reaching nearly 40%, as these objects possess distinct edges or corners with clear structural features, enabling SIFT to capture more pronounced local characteristics. Accuracy on SVHN dataset is marginally higher at 47.75%, as numbers possess more pronounced edges relative to images and exhibit greater relative contrast against backgrounds. Training durations for the two datasets were 125.3 minutes and 445.4 minutes respectively. Although SIFT demonstrates high feature extraction efficiency, the linear classifier of SVM significantly impedes training speed, resulting in markedly lower efficiency compared to deep learning algorithms.

Overall, with both datasets operating at 32×32 resolution, SIFT fails to generate sufficient scale-level pyramids. Furthermore, local regions occupy disproportionately large portions of the image, resulting in significant information loss. Within the bag-of-words model, the absence of positional relationships and relative scale prevents differentiation between distance and near object features and spatial distribution variations, further contributing to information loss. When selecting 200 visual vocabulary terms, the information loss rate exceeded 90%. For the linear SVM decision function, since x itself lacks sufficient discriminative information, once accuracy reaches a certain upper limit, further optimization of w proves ineffective in improving accuracy.

4. Explanation and comparison

For CiFAR-10 dataset and SVHN dataset, the accuracy of deep learning is significantly higher than that of traditional computer vision. Deep learning focusses on autonomous learning, through hierarchical structures, it can learn the internal relationships of objects, such as judging the category of objects as automobile through wheels. The deepening of the number of channels can learn the semantic characteristics of the CiFAR-10 dataset, and broadening can learn the detailed characteristics of the SVHN dataset. The model independently summarises and summarises the characteristics of objects or numbers through a large amount of data, so the accuracy rate is relatively high. Compared with traditional computer vision, local features are learned only through SIFT, and SVHN's messy street background and smaller pixels will seriously affect the learning results of SIFT. In the more complex CiFAR-10 data set, the morphological changes of objects make it more difficult to learn the word bag model. By describing fragmented local textures, naturally not get the same accuracy as deep learning.

5. State of the art

The fundamental difference between robot vision and computer vision lies in the result

of image transformation. Computer vision is transformed into information and robot vision is transformed into action. Therefore, closed-loop interaction must be considered. If active perception is not carried out, the error accumulation will increase in subsequent actions (N. Sünderhauf et al., 2018). Therefore, RT-2 uses the Internet-level large-scale visual-language pre-training model to guide the generation of robot action sequences and create the VLA paradigm (B. Zitkovich et al., 2023). On this basis, π0 (K. Black et al., 2024) and TinyVLA (J. Wen et al., 2025) combine stream matching and diffusion strategy decoder to VLA, which can perform continuous and high-precision actions at high frequency, and the success rate in complex and long-sequence tasks has increased significantly . In addition, FP2AT extends the observation of robots only in 2D images to 3D, and uses multi-scale three-dimensional vision to integrate attention mechanisms to improve the ability of visual perception and action planning (Y. Liu et al., 2025). The practice of OK-Robot provides a possible direction for the VLA paradigm in the future. The VLA model is used for high-level abstract understanding and planning. The special modules in OK-Robot, such as grabbers, are used for reliable low-level execution (P. Liu et al., 2024). Although cutting-edge technologies such as VLA solve problems in data efficiency and other dimensions, But it is still impossible to solve all system-level engineering challenges.

6. References

Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., ... & Zhilinsky, U. (2024). $\pi_0 $: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*.

Liu, P., Orru, Y., Vakil, J., Paxton, C., Shafiullah, N. M. M., & Pinto, L. (2024). Ok-robot: What really matters in integrating open-knowledge models for robotics. *arXiv preprint arXiv:2401.12202*.

Liu, Y., Liu, S., Chen, B., Yang, Z. X., & Xu, S. (2025). Fusion-Perception-to-Action Transformer: Enhancing Robotic Manipulation with 3D Visual Fusion Attention and Proprioception. *IEEE Transactions on Robotics*.

Sünderhauf, N., Brock, O., Scheirer, W., Hadsell, R., Fox, D., Leitner, J., ... & Corke, P. (2018). The limits and potentials of deep learning for robotics. *The International journal of robotics research*, *37*(4-5), 405-420.

Wen, J., Zhu, Y., Li, J., Zhu, M., Tang, Z., Wu, K., ... & Tang, J. (2025). Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*.

Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., ... & Han, K. (2023, December). Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning* (pp. 2165-2183). PMLR.

Appendix

```
## Deep_Learning_CiFAR-10
def create_cifar10_cnn_model():
    model = tf.keras.Sequential()

    model.add(tf.keras.layers.Conv2D(32,     (3,     3),     padding='same',
input_shape=(32, 32, 3)))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.Activation('relu'))
    model.add(tf.keras.layers.Conv2D(32, (3, 3), padding='same'))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.Activation('relu'))
    model.add(tf.keras.layers.MaxPooling2D((2, 2)))
    model.add(tf.keras.layers.Dropout(0.2))

    model.add(tf.keras.layers.Conv2D(64, (3, 3), padding='same'))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.Activation('relu'))
    model.add(tf.keras.layers.Conv2D(64, (3, 3), padding='same'))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.Activation('relu'))
    model.add(tf.keras.layers.MaxPooling2D((2, 2)))
    model.add(tf.keras.layers.Dropout(0.3))

    model.add(tf.keras.layers.Conv2D(128, (3, 3), padding='same'))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.Activation('relu'))
    model.add(tf.keras.layers.Conv2D(128, (3, 3), padding='same'))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.Activation('relu'))
    model.add(tf.keras.layers.MaxPooling2D((2, 2)))
    model.add(tf.keras.layers.Dropout(0.4))

    model.add(tf.keras.layers.Flatten())
    model.add(tf.keras.layers.Dense(256))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.Activation('relu'))
```

```
    model.add(tf.keras.layers.Dropout(0.5))

    model.add(tf.keras.layers.Dense(num_classes, activation='softmax'))

    return model

## Deep_Learning_SVHN
def create_svhn_cnn_model():
    model = tf.keras.Sequential([
        layers.Conv2D(64, 3, padding='same', kernel_initializer='he_normal',
input_shape=(32, 32, 3)),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Conv2D(64,                3,                padding='same',
kernel_initializer='he_normal'),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.MaxPooling2D(2),
        layers.Dropout(0.2),

        layers.Conv2D(128,               3,                padding='same',
kernel_initializer='he_normal'),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Conv2D(128,               3,                padding='same',
kernel_initializer='he_normal'),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.MaxPooling2D(2),
        layers.Dropout(0.3),

        layers.Conv2D(256,               3,                padding='same',
kernel_initializer='he_normal'),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Conv2D(256,               3,                padding='same',
kernel_initializer='he_normal'),
        layers.BatchNormalization(),
```

```
        layers.Activation('relu'),
        layers.MaxPooling2D(2),
        layers.Dropout(0.4),

        layers.Flatten(),
        layers.Dense(512, kernel_regularizer=regularizers.l2(1e-4)),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Dropout(0.5),

        layers.Dense(256, kernel_regularizer=regularizers.l2(1e-4)),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Dropout(0.5),

        layers.Dense(10, activation='softmax')
    ])
    return model

## Traditional_Computer_Vision
def extract_sift_descriptors(images, desc="Extracting SIFT Descriptors"):
    sift = cv2.SIFT_create()
    all_descriptors = []
    image_descriptors_list = []
    valid_indices = []

    for i, img in enumerate(tqdm(images, desc=desc)):
        gray_img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
        _, des = sift.detectAndCompute(gray_img, None)

        if des is not None and des.shape[0] > 0:
            all_descriptors.append(des)
            image_descriptors_list.append(des)
            valid_indices.append(i)
        else:
            image_descriptors_list.append(None)

    if all_descriptors:
```

```
        all_descriptors_concatenated = np.concatenate(all_descriptors)
    else:
        all_descriptors_concatenated = np.array([])

    return      image_descriptors_list,      all_descriptors_concatenated,
valid_indices

def create_bow_features(image_descriptors, kmeans_model):
    k = kmeans_model.n_clusters
    image_features = []

    for des in image_descriptors:
        labels = kmeans_model.predict(des)
        hist, _ = np.histogram(labels, bins=k, range=(0, k))
        image_features.append(hist)

    return np.array(image_features)

def run_traditional_cv(dataset_name, visual_words_k):
    training_summary['visual_words'] = visual_words_k
    training_summary['phase_times'] = {}

    start_time = time.time()
    X_train, y_train, X_test, y_test = load_and_extract_data(dataset_name)
    training_summary['phase_times']['data_loading']    =    time.time()    -
start_time

    start_time = time.time()
    train_descriptors_list,  all_train_descriptors,  train_valid_indices  =
extract_sift_descriptors(X_train)
    training_summary['phase_times']['sift_train']    =    time.time()    -
start_time

    y_train_filtered = np.array(y_train)[train_valid_indices].tolist()
    train_descriptors_list_filtered = [train_descriptors_list[i] for i in
train_valid_indices]

    if len(y_train_filtered) == 0:
```

```
        print("Error: No SIFT descriptors could be extracted from the training set.")
        return
    print(f"Valid training samples after SIFT extraction: {len(y_train_filtered)}")
    start_time = time.time()

    kmeans = MiniBatchKMeans(
        n_clusters=visual_words_k,
        random_state=42,
        n_init=10,
        verbose=0
    )
    kmeans.fit(all_train_descriptors)
    training_summary['phase_times']['kmeans'] = time.time() - start_time

    start_time = time.time()
    X_train_bow = create_bow_features(train_descriptors_list_filtered, kmeans)
    training_summary['phase_times']['bow_train'] = time.time() - start_time

    start_time = time.time()
    test_descriptors_list, _, test_valid_indices = extract_sift_descriptors(X_test)
    training_summary['phase_times']['sift_test'] = time.time() - start_time

    y_test_filtered = np.array(y_test)[test_valid_indices].tolist()
    test_descriptors_list_filtered = [test_descriptors_list[i] for i in test_valid_indices]

    X_test_bow = create_bow_features(test_descriptors_list_filtered, kmeans)

    start_time = time.time()
    scaler = StandardScaler().fit(X_train_bow)
    X_train_scaled = scaler.transform(X_train_bow)
    X_test_scaled = scaler.transform(X_test_bow)
    training_summary['phase_times']['scaling'] = time.time() - start_time
```

```
    start_time = time.time()

    svm = SVC(
        kernel='linear',
        C=1.0,
        random_state=42
    )
    svm.fit(X_train_scaled, y_train_filtered)
    training_summary['phase_times']['svm_training']    =    time.time()    -
start_time
```